

Lab 4: Working with threads in Linux

1. Create a thread that prints numbers from 1 to N, one per line. N is entered from keyboard. The main thread should wait for this thread to finish before printing "Main thread exiting."
2. Write a program where a new thread computes the sum of numbers from 1 to N and returns the result. N is entered from keyboard. The main thread should retrieve this result and print it.
3. Write a program that creates three threads. Each thread should print a unique message along with its thread ID. The main thread should print "Main thread is running". The program should wait for all threads to finish before exiting.
4. Complete below two programs: Program 1 create a thread modifying a global variable. Program 2 create a process modifying the same global variable. Afterthat, executing two programs. What are the value of global variables in two programs? Explain why.

Program 1	Program 2
<pre> #include <pthread.h> #include <stdio.h> #include <stdlib.h> int sharedVar = 5; // Shared among threads void *threadFunc(void *arg) { //Write code change value of global variable here } int main() { pthread_t thread; // Write code to create thread here //Write code to wait for thread to terminate here printf("Main: sharedVar = %d\n", sharedVar); return 0; } </pre>	<pre> #include <stdio.h> #include <stdlib.h> #include <unistd.h> int sharedVar = 5; // Separate copy for each process int main() { //Write code to create child process here if (pid == 0) { // Child process //Write code change value of global variable here printf("Child Process: sharedVar = %d\n", sharedVar); exit(0); } else { //Write code to wait for child process to terminate here printf("Parent Process: sharedVar = %d\n", sharedVar); } } </pre>

	return 0; }
--	----------------

5. Write a program that:
 - Defines a global counter.
 - Creates two threads, one thread incrementing the counter 100 times; one thread decreasing the counter 50 times
 - Uses a mutex to prevent race conditions.
 - Prints the final counter value.
6. A barrier is a synchronization mechanism that makes all threads wait until a specific number of threads have reached a common point.

Write a program that:

- Creates N threads (e.g., 4 threads).
- Each thread should print "Thread X reached the barrier", then wait at the barrier.
- The last arriving thread should print "All threads reached the barrier! Releasing them..." and allow all threads to proceed.
- Each thread should print "Thread X passed the barrier" after being released.

Hint: Using a mutex to protect a counter and a condition variable to signal all threads when the last thread arrives.

7. Write a producer-consumer program using a mutex and a condition variable.

The program should:

- Use a shared buffer (an integer array of size n).
- Have a producer thread that:
 - + Generates numbers and adds them to the buffer.
 - + Waits if the buffer is full.
- Have a consumer thread that:
 - + Removes and prints numbers from the buffer.
 - + Waits if the buffer is empty.